

Reflection & Plugins in C#

SWEN1 | BIF3

Table of Contents

- Reflection in C#
- Plugins in C#



Reflection in C#

Introduction to Reflection

In computer science, reflection is the ability of

- a process to examine,
- introspect, and
- modify
  its own structure and behavior. (Wikipedia)

Use reflection to

- create objects of classes which are injected from outside (plugins),
- to check the outer interface of a class (call method “print()” if it exists) or
- to find classes (find all classes that implement the interface AudioPlugin in a library).

Examples

Obtain the Type of an Object:

```
// Using typeof operator
Type = typeof(MyClass);
// Using GetType() method on an instance
MyClass obj = new MyClass();
Type = obj.GetType();
```

Retrieve Type Information:

```
Type = typeof(MyClass);
MemberInfo[] members = type.GetMembers();
foreach (MemberInfo member in members)
{
   Console.WriteLine(member.Name);
}
```

Examples

Invoke Methods dynamically:

```
Type = typeof(MyClass);
object obj =
    Activator.CreateInstance(type);

MethodInfo method =
    type.GetMethod("MyMethod");
method.Invoke(obj, null);
```

Access and modify Properties:

```
PropertyInfo property =
    type.GetProperty("MyProperty");
object value = property.GetValue(obj);
property.SetValue(obj, newValue);
```

Extended Sample – find class/members by attribute

```
class MethodAttr : System.Attribute {
    public string Method;
    ...
}
class PathAttr : System.Attribute {
    public string Path;
    ...
}

...

var endpointClass =
    Assembly.GetExecutingAssembly().GetTypes().Where(
        type => type.GetCustomAttribute<PathAttr>()?.Path == _request.Path).Single();
var endpointMethod =
    endpointClass.GetMethods().Where(
        method => method.GetCustomAttribute<MethodAttr>()?.Method ==
            _request.Method).Single();

_response.Content = (string)endpointMethod.Invoke(
    Activator.CreateInstance(endpointClass), null);
```



Plugins in C#

Plugins

- define an interface in your base project
- search for libraries (DLL)
- get all types implementing the interface
- create an instance of that type dynamically
- call the instance used by the interface
- trust and believe

Plugin Sample

Interface

```
namespace PluginBase
{
    public interface ICommand
    {
        string Name { get; }
        string Desc { get; }

        int Execute();
    }
}
```

Type

```
namespace HelloPlugin
{
  public class HelloCommand : ICommand
  {
    public string Name { get => "hello"; }
    public string Desc { get =>
       "Displays hello message."; }

    public int Execute()
    {
      Console.WriteLine("Hello !!!");
      return 0;
    }
  }
}
```

Load Assemblies

```
class PluginLoadContext : AssemblyLoadContext
{
  private AssemblyDependencyResolver _resolver;
  public PluginLoadContext(string pluginPath) {
    _resolver = new AssemblyDependencyResolver(pluginPath);
  }
  protected override Assembly Load(AssemblyName assemblyName) {
    string assemblyPath = _resolver.ResolveAssemblyToPath(assemblyName);
    return assemblyPath != null ? LoadFromAssemblyPath(assemblyPath) : null;
  }
  // ...
}

PluginLoadContext loadContext = new PluginLoadContext(pluginLocation);
Assembly assembly = loadContext.LoadFromAssemblyName(
    new AssemblyName(Path.GetFileNameWithoutExtension(pluginLocation)));
```

Load Types

```
foreach (Type type in assembly.GetTypes())
{
  if (typeof(ICommand).IsAssignableFrom(type))
  {
    ICommand result = Activator.CreateInstance(type) as ICommand;
    if (result != null)
    {
      return result;
    }
  }
}
```